System Architecture: Beginner to Advanced

Beginner (Fundamentals)

What is System Architecture?

System architecture is the high-level "blueprint" of a software system – a conceptual model that defines the system's structure, behavior, and components [1] [2]. It's like the architectural plan for a building: it lays out *what pieces go where* and *how they interact* so the system (like a house) works as intended. For example, one analogy calls system architecture "the blueprint for a building… defining how all parts of a computer system or software work together" [2]. In practice, a good architecture ensures a system is coherent and can evolve; without it, building software is like "building a house without a plan" [2].

Goals of System Architecture

A system is built to meet key non-functional goals. **Scalability** means handling growth – adding more users or data by scaling up (bigger machines) or out (more machines). **Availability** ensures the system stays up (even if parts fail). **Maintainability** means the system can be changed or extended easily (through modular design). **Security** protects data and users (confidentiality, integrity). **Cost-effectiveness** means keeping resource and operational costs reasonable. These goals often trade off against each other (e.g. higher availability usually costs more [3]). In short, architects must balance **scalability, availability, performance, security,** and **cost** when designing a system [4] [3].

Goal	Description
Scalability	Ability to handle increased load by scaling (vertical or horizontal). Ensures the system gracefully grows under more users or data.
Availability	System remains up and responsive despite failures. Achieved via redundancy and failover (multiple servers, health checks, auto-restart) [4].
Maintainability	Ease of updating or fixing the system. Modular code and clear interfaces help future development.
Security	Protects against unauthorized access or attacks. Built-in via measures like authentication, encryption, least privilege.
Cost	Control over resource use and operational costs. Trade-offs with performance: "more availability" usually means higher hardware/ops cost [3].

Architecture Views

Complex systems are described via different **architecture views**. Each view addresses certain concerns:

- **Logical View** – focuses on the abstract structure of the system (components, modules, layers, and their relationships). For example, UML diagrams showing how classes or services relate [5].
- **Process View** – shows runtime behavior, including processes or threads and their interactions (e.g. sequence or activity diagrams). It covers concurrency and data flows at run-time [6].
- **Physical (Deployment) View** – shows how the system is deployed on hardware. It includes servers, network topology, and how software components map to machines [7].
- **Data View** – describes the data model and data flow. It explains how data is stored, structured, and moves through the system. A *data architecture* blueprint defines how data is collected, transformed, and distributed [8].

Each view is like a different blueprint: e.g., the **context** diagram (C4 level 1) shows the system's boundaries with users and external systems [9]; the **container** diagram (C4 level 2) shows major components (apps, databases, etc.); and so on [9]. Together, these views ensure all stakeholders (developers, operators, business leads) understand the system.

Client–Server Model and APIs

Most modern systems follow a **client–server** model: a client (e.g. browser or mobile app) sends a request over the network to a server, which processes it and returns a response [10]. In this model, the client initiates communication ("request"), and the server answers ("response") [10]. For example, a web browser (client) requests a webpage (GET /index.html) from a web server, which replies with HTML. Servers can be anything from a simple web server to a complex cluster handling many clients.

An **API** (Application Programming Interface) is the set of rules and endpoints through which clients talk to a server. APIs define *how* to request data or actions (e.g. REST, gRPC, GraphQL). In essence, an API is a contract: "Send me this request format, and I'll give you that response" [11]. For example, a REST API might let clients `POST /users` to create a user or `GET /items` to fetch items. Designing clean, versioned APIs is a core part of system architecture, as it defines interactions between components.

Core Concepts

Servers: Physical, Virtual Machines, Containers

A **physical server** (or "bare metal") is actual hardware (CPU, RAM, disks) running an operating system and applications. It's non-virtualized, so you use the machine's full resources for one OS. In contrast, a **virtual machine (VM)** is a software abstraction: it emulates a full computer, complete with its own OS, on top of a host. One physical server can run many VMs via a hypervisor (e.g. VMware, KVM). Each VM is like a separate computer, but VMs incur overhead because they virtualize hardware (each has its own OS) [12].

Containers are lighter-weight than VMs. A container packages an application and its dependencies (libraries, config files) into a single unit. Containers share the host OS kernel and isolate the app's user-space. Docker is a popular container engine; it lets you run many containers on a host without the overhead of full VMs [12]. In summary: physical servers are hardware; VMs run full OS images (better isolation, some

overhead); containers (Docker) share the OS (more efficient, faster startup) [12] [13]. Containers have become fundamental building blocks in modern architectures (often orchestrated by Kubernetes).

Networking (IP, DNS, Ports, Reverse Proxy, CDN)

IP addresses uniquely identify devices on a network (e.g. `192.168.1.10` or IPv6). The **Domain Name System (DNS)** is like the Internet's phone book: it translates human-friendly names (e.g. `www.example.com`) into IP addresses [14]. DNS is hierarchical and distributed; for instance, when you type a domain, DNS servers reply with the numeric IP that web servers listen on [14].

Ports are numeric endpoints on an IP address (0–65535) that identify specific services. For example, HTTP typically uses TCP port 80 (or 443 for HTTPS). A single server (IP) can host many services by listening on different ports or interfaces.

A **reverse proxy** is a network component (like Nginx or HAProxy) that sits in front of one or more servers. Clients connect to the proxy, and it forwards requests to the appropriate backend server. Reverse proxies often perform load balancing, SSL termination, URL routing, and security (e.g. filtering). They present a unified endpoint to clients while distributing traffic across servers.

A **CDN (Content Delivery Network)** is a globally distributed cache of static content (images, videos, scripts). CDNs (like Cloudflare, Akamai) store copies of content at edge locations near users [15]. When a user requests a large asset, the CDN serves it from the nearest edge server instead of the origin, greatly reducing latency. CDNs dramatically speed up content delivery for websites and APIs by caching resources closer to users [15].

Databases: SQL vs NoSQL, ACID vs BASE, Indexing, Normalization

- **SQL vs NoSQL:** SQL (relational) databases store data in tables with a fixed schema (rows and columns). They enforce relationships via foreign keys and support ACID transactions. They scale mostly vertically (bigger machines). NoSQL (non-relational) databases include key-value stores, document stores, wide-column, or graph databases. They usually have flexible schemas (or schema-less) and scale horizontally across many servers [16] [17]. In general, SQL is chosen for structured data and complex queries, while NoSQL is chosen for very high-scale or unstructured data. For example, an RDBMS (MySQL, PostgreSQL) might suit a billing system, whereas a NoSQL DB (Cassandra, MongoDB) might suit storing massive logs or user events [16] [18].

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Schema	Fixed (predefined tables)	Flexible or schema-less
Data Model	Tables, rows, joins	Key-value, document, column-family, graph, etc.
Scaling	Vertical (scale-up)	Horizontal (scale-out) across nodes [16]
Transactions	ACID-compliant (strong)	Often BASE (eventual) for high availability [16]

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Use Cases	Traditional apps, OLTP, complex queries	Big data, analytics, caching, real-time feeds

- **ACID vs BASE:** These terms describe transactional consistency. **ACID** (Atomicity, Consistency, Isolation, Durability) is the property of traditional databases: each transaction is all-or-nothing, preserves consistency rules, and once committed, stays durable [19]. For example, money transfer should either complete both debit and credit or none (atomic), maintaining consistency [19]. **BASE** (Basically Available, Soft state, Eventual consistency) is often used in NoSQL systems: the system is designed to be always available and partition-tolerant, but immediate consistency may be sacrificed. With BASE, updates eventually propagate and replicas converge. In practice, BASE means "write now, sync later." For instance, a social network might show a "like" count that is temporarily stale but will catch up (eventual consistency) [20].

- **Indexing:** An *index* is a data structure (like a B-tree) on database columns to speed up queries. Without indexes, a database must scan every row (full table scan) to find matches. With an index, the DB can quickly locate rows by looking up the key in the index structure [21]. For example, an index on `email` column lets queries like `WHERE email='user@example.com'` run in logarithmic time instead of scanning all users. Indexes improve read/query performance, but they consume space and slow down writes (as indexes must be updated). In summary: **create indexes on columns you search/sort on** to speed queries [21] [22].

- **Normalization:** Database normalization is the process of organizing tables to reduce redundancy and ensure data integrity [23]. For instance, rather than storing a user's address in multiple tables, you'd have an `Addresses` table and refer to it by ID. Normal forms (1NF, 2NF, 3NF, etc.) provide rules to break tables into smaller linked tables (via keys) so updates happen in one place. The benefit is avoiding inconsistencies and wasted space [23]. However, over-normalization can hurt performance (because many JOINs), so some systems denormalize carefully for speed.

Distributed System Basics

CAP Theorem

The **CAP theorem** states that in the presence of a network partition, a distributed system can guarantee **either** Consistency (C) or Availability (A), but not both [24]. Here *Consistency* means every read sees the most recent write (single-copy illusion), and *Availability* means every request receives a (non-error) response. For example, if a data-center link breaks, you must choose: either stop serving reads (sacrificing Availability) until data syncs (ensuring Consistency), or continue serving possibly stale data (sacrificing immediate Consistency) [24] [25]. Partition Tolerance (P) means the system continues operating despite network failures, and in practice you almost always need P in a distributed system (so the real choice is CA vs CP vs AP tradeoff).

Real-world examples: Google Spanner (a distributed SQL DB) uses highly synchronized clocks and 2-phase commits to give strong consistency (CP) across zones [26]. Amazon's Dynamo (and many NoSQL stores like Cassandra) choose availability (AP): they always answer requests, but replicas converge to consistency later

(eventual consistency) [26] [27]. In other words, Dynamo prioritizes being up (even if data is briefly inconsistent), while Spanner sacrifices availability on partitions to keep data strongly consistent.

Consistency Models (Strong vs Eventual)

- **Strong consistency** means a read reflects the result of all prior writes. Every client sees the same data immediately after a write completes. Achieving strong consistency usually requires coordination (locking or consensus) between replicas, which can add latency. In practice, strong consistency is used for critical data (e.g., banking ledgers) but can limit availability if nodes fail.

- **Eventual consistency** means that given enough time without new updates, all replicas will converge to the same value. In this model, a read might return stale data if a recent update hasn't propagated yet. For example, if one region updates a follower count from 100 to 101, another region might still see 100 for a short while [28]. Eventually the update trickles through and both show 101. This approach allows high availability and partition tolerance, at the cost of brief inconsistency. Many large-scale systems (like DNS or many NoSQL stores) use eventual consistency to remain highly available.

Latency and Fault Tolerance

Latency refers to the time delay between a request and its response. System architects strive to minimize latency (fast responses) for good user experience. Techniques include caching, data locality (placing compute close to data and users), and efficient algorithms [29]. For instance, keeping user data cached in-memory in a nearby data center minimizes hops and thus latency [29].

Fault Tolerance is the ability to continue operating despite failures (server crashes, network glitches). Key strategies are redundancy and automated recovery. For example, using multiple replicas of a service or database ensures that "no single failure takes it down" [4]. Sophisticated fault-tolerance involves retries, circuit breakers, and health checks [4]. Health checks can detect a bad server and remove it from service; retries or replays can recover transient errors. Overall, a fault-tolerant design uses replicated components and failure-detection logic so that individual faults become imperceptible to end users [4].

Intermediate Concepts

Monolith vs Microservices

A **monolithic** architecture is built as one cohesive unit: all functionality (UI, business logic, data access) lives in a single codebase and is deployed together. It's simpler to develop at first but can become unwieldy at scale. A **microservices** architecture splits functionality into small, independent services (each with its own process and data) [30]. Microservices allow independent development and scaling of components.

However, microservices add complexity: network communication, distributed deployment, testing, and data consistency are harder. Thus **monoliths are not wrong** when an application is small or a team is small. You might *avoid* microservices if your project is simple or the overhead isn't justified. As one source notes: "A monolithic app is a single unified unit, while microservices are a collection of smaller deployable services" [30]. Starting as a monolith and refactoring later is common when needs grow.

Load Balancers (Layer 4 vs Layer 7)

A **load balancer (LB)** distributes incoming network traffic across multiple backend servers to improve reliability and throughput. LBs can operate at different OSI layers:

- **Layer 4 (Transport)** load balancing uses IP address and port (TCP/UDP) information. It routes packets based on source/destination IP and port, without looking at the actual payload. This is fast and protocol-agnostic. Example: an LB that only knows "send traffic for port 80 to server 1 or 2 in round-robin" [31].

- **Layer 7 (Application)** load balancing inspects the application layer (typically HTTP) and makes routing decisions based on content (URLs, headers, cookies). This allows smart routing like "route `/images/*` to image servers" or A/B testing. It can also do SSL termination. However, inspecting application data adds overhead. For example, an L7 LB can look at `Host:` or URI to decide which server cluster gets the request [31].

In short, **L4 LB** is fast and general (just TCP ports) [31], while **L7 LB** is more flexible (knowing HTTP/HTTPS details) but more complex. Many products (like Nginx or HAProxy) can operate in either mode.

Caching

Caching stores copies of data to serve future requests faster. There are multiple types:

- **Client (Browser) Cache:** Browsers cache static files (HTML, CSS, JS, images) per HTTP headers (Cache-Control) so repeat visits are faster. DNS resolvers cache name lookups, too. This happens on end-user devices or proxies.

- **CDN Cache:** As noted, CDNs cache static content at edge servers worldwide [15], reducing load on origin servers and speeding delivery.

- **Application Cache (In-Memory):** Within an application cluster, you can cache data in memory (e.g. using Redis, Memcached). For instance, you might cache results of expensive DB queries or sessions in a distributed cache. This drastically cuts response time for repeated queries [32].

Caching Strategies:

- **Cache-Aside (Lazy Loading):** The application first checks the cache; on a *cache miss*, it loads data from the database and then populates the cache [33]. This keeps the cache simple but means the first request is slower.

- **Write-Through:** Every write updates both the cache and the backing store simultaneously [34]. This keeps cache and DB in sync, but every write is slower.

- **Write-Back:** Writes go to the cache first, and the cache asynchronously writes to the database [35]. This makes writes very fast, but has risk: if the cache fails before persisting, data loss can occur.

A major challenge is **cache invalidation** (knowing when cached data is stale). One famous quip is "There are only two hard things in computer science: cache invalidation and naming things" [36] . In practice, we solve invalidation by setting TTLs (time-to-live) on entries or updating/evicting cache on writes. As one explanation notes, cache invalidation must happen when source data changes, or else stale (incorrect) data may be served [37] . Properly balancing freshness versus performance is a key design decision.

Message Queues and Streaming

Message queues decouple producers and consumers of data. Systems post messages to a queue and other services pull them. There are two paradigms:

- **Traditional Queue (e.g. RabbitMQ):** Acts like a queue of tasks. Producers enqueue messages, and one or more consumers dequeue them. Once a message is consumed, it's gone. Ideal for work distribution (e.g. background jobs). RabbitMQ is a typical broker: it can route messages via exchanges, supports complex routing, but once a consumer ACKs a message, that message is removed.

- **Log/Stream (e.g. Kafka):** Kafka treats messages as an append-only log. Producers publish messages to a *topic* (like an event stream). Consumers maintain their own offset (pointer) and can read messages independently. In Kafka, messages are not removed upon one consumer reading; multiple consumers can read the same stream at different times. Kafka is designed for high-throughput, distributed streaming.

A key difference: RabbitMQ (queue) delivers each message to one consumer (or a competing group) and then discards it [38] , whereas Kafka (stream) retains all messages for a configured retention period and allows multiple consumers (or consumer groups) to reprocess them [38] . In short, **queues** are for tasks needing one-time consumption; **streams** are for broadcast events and event sourcing.

Use cases:

- *RabbitMQ (Queue)*: Asynchronous tasks like sending email, thumbnail generation, pipeline jobs. Great for point-to-point messaging where exactly-once processing is needed.
- *Kafka (Stream)*: Event logging, real-time analytics, and publish/subscribe. For example, websites write user activity to Kafka; multiple downstream services (metrics, search indexing, alerts) consume the same stream independently [39] .

Both can handle retries and delivery guarantees, but Kafka shines when you need to replay historical events or handle very high throughput, while RabbitMQ shines for reliable message delivery and flexible routing.

Advanced Concepts

Database Scaling: Replication and Sharding

- **Replication:** Copying the same data to multiple database instances. Commonly, one *master* handles writes and many *read replicas* (slaves) handle reads. Replication increases availability and read throughput: if a replica goes down, others still serve reads. It also allows distributing read queries

geographically (close to users). However, writes still go to the master, which can become a bottleneck [40] [41].

- **Sharding (Partitioning):** Splitting a database into pieces (shards) across multiple servers. Each shard holds a subset of the data (e.g. users 1–10M on shard1, 10M–20M on shard2). Sharding scales writes by distributing them, at the cost of complexity (joins across shards become hard). A classic example: a social network may shard posts by user ID hash. Good sharding balances load; poor sharding (e.g. hot keys) causes skew.

In summary: replication is about **copying** data for read-scale and fault-tolerance [41]; sharding is about **splitting** data to scale writes [40]. Many systems use both: e.g. each shard is replicated. A hybrid strategy might have 10 shards with 3 replicas each, covering 30 servers total.

Distributed Transactions and Saga Pattern

In microservices or distributed databases, a single business operation may span multiple services/databases. Traditional two-phase commit (2PC) is often too slow or brittle. The **Saga pattern** offers an alternative: it breaks a transaction into a series of local transactions, each with a compensating (undo) transaction [42].

For example, consider an order process: a saga might debit a bank account (local transaction), then reserve an item in inventory (local transaction). If the second step fails, the saga triggers a compensating action: credit back the account. Each step publishes an event to trigger the next step, and any failure triggers compensations for all previous steps [42]. This eventual approach avoids global locking; the system achieves consistency through rollbacks as needed. The microservices.io description: “Each local transaction updates the database and publishes a message. If one fails, the saga executes compensating transactions to undo previous changes” [42].

Consensus Algorithms (Raft)

When multiple nodes must agree on a value (e.g. which server is the leader, or the order of updates), consensus algorithms are used. **Raft** is a popular consensus protocol that is easier to understand than Paxos. Raft elects one node as a **leader**; this leader takes client writes and replicates a log of entries to follower nodes. If the leader fails, the cluster holds a quick election to choose a new leader [43]. The key idea: only the leader appends new entries, and followers merely replicate. Once a majority of followers have written the entry, it’s committed. This ensures consistency: all nodes agree on the same sequence of entries. In short, Raft provides fault-tolerant, replicated state machines using a leader-based log replication [43].

Event-Driven Architecture (EDA)

An event-driven architecture revolves around **events**: significant changes in state (e.g. “OrderCreated”, “ItemShipped”). In EDA, services **publish events** when things happen, and other services **subscribe** to those events to react. This decouples producers and consumers: the producer just emits an event, without knowing who consumes it [44] [45].

Event flow: A typical flow is: A service (producer) performs an operation and publishes an event message (often to a broker or message bus). The broker routes the event to interested consumers. Consumers then

take actions (update database, call other services) in response. For example, when a user places an order, the Order Service publishes an `OrderPlaced` event to a message bus; downstream Inventory and Shipping services listen and act on that event.

Events have a key, payload, timestamp, and possibly metadata [46]. They can either carry full state (e.g. the new order details) or just an identifier with details fetched separately. Event types include notifications, state change events, integration events, etc. The flow is asynchronous and often real-time, enabling responsive, scalable systems. As IBM notes, EDA treats the system as a series of event producers and consumers, with "publish, capture, process, and store events" at its core [44] [45].

CQRS (Command-Query Responsibility Segregation) and Event Sourcing

CQRS separates reads (queries) from writes (commands) into different models or even databases [47]. The idea is that read operations (which often require denormalized, optimized views) are completely separate from write operations (which enforce business rules). You might have one data store optimized for fast writes and another (or multiple) optimized for fast reads. CQRS enables independent scaling of read and write sides and can simplify complex logic by having distinct paths for commands vs. queries [47].

Event Sourcing is a pattern where the system's state is not stored as a single snapshot, but as an immutable sequence of events [48]. Each change (e.g. "deposit $10", "withdraw $5") is an event recorded in an append-only log. The current state is reconstructed by replaying all events. This provides a full audit log and simplifies consistency (since the event log is the one truth). Event Sourcing is often used *with* CQRS: writes are event streams (source of truth) and reads are built as views projected from those events. As the GeeksforGeeks article says, CQRS decouples read/write models, while event sourcing "stores state as a sequence of events" [48]. The combination allows high scalability, auditability, and flexibility (e.g. you can "time travel" the state by replaying events).

Service Mesh (Istio)

A **service mesh** is an infrastructure layer for handling service-to-service communication in a microservices architecture. It consists of a control plane and a data plane. For example, Istio is a popular service mesh. Istio inserts a lightweight proxy (often Envoy) alongside each service instance as a sidecar. These sidecars intercept all network traffic to/from the service. The mesh's control plane configures and manages these proxies, so that developers don't have to handle networking, security, or observability in each microservice.

Istio's purpose is to manage and secure communication between microservices without changing application code [49]. It provides out-of-the-box features like traffic routing, load balancing, retries, circuit breaking, mutual TLS, and telemetry across the cluster. In Istio, **Istiod** (control plane) distributes configuration and certificates, while each service's Envoy proxy (data plane) enforces policies on the fly [49] [50]. In effect, Istio is a "smart, configurable networking layer" that decouples operational concerns from business code [49]. In an architecture diagram, a service mesh is often drawn as the network layer overlaid on all service instances (with an icon for Envoy or Istio).

Data Engineering

Batch vs Stream Processing

- **Batch Processing:** Collects large volumes of data and processes them as groups at scheduled times. Think of nightly payroll or monthly reports. It's offline processing where latency isn't critical. For example, every hour or day, process all new log files through a MapReduce job. This is efficient for huge datasets and allows sophisticated computation, but is inherently delayed [51].
- **Stream (Real-time) Processing:** Handles data continuously and incrementally as it arrives [52]. Data is processed in motion, enabling near-real-time analytics. For example, an e-commerce site might process clickstream or purchase events as they occur to update recommendations instantly. Stream processing requires frameworks like Apache Flink, Kafka Streams, or Spark Streaming, and it prioritizes low latency over batch throughput.

DataCamp summarizes: "Batch processing is a method in which large volumes of collected data are processed in chunks or batches [51], whereas stream processing deals with data continuously and incrementally [52]." Batch is simpler and cost-effective for historical data analysis; streaming is essential for timely insights (fraud detection, monitoring) at the expense of complexity.

Lambda vs Kappa Architecture

These are two big-data architectural patterns combining batch and real-time processing:

- **Lambda Architecture:** Splits processing into two parallel layers [53]. The *batch layer* handles large-scale historical data in batches (high throughput, high accuracy). The *speed layer* handles real-time data quickly but with possibly less accuracy. A *serving layer* merges their outputs, giving combined results. Lambda is robust (fault-tolerant, can recompute entire history), but complex because you maintain two pipelines [53]. It's useful when you need both real-time responsiveness and exact historical computation.

- **Kappa Architecture:** Uses a single stream processing pipeline for both real-time and historical data [54]. All data is treated as an append-only log. To "reprocess" historical data, you simply replay the log through the same streaming system [55]. Kappa is simpler (no separate batch layer) and works well if your use case can use streaming for everything. However, it may sacrifice the highest accuracy that a dedicated batch job could provide.

In short, Lambda = batch *and* speed layers; Kappa = only streaming (log) layer [56] [57]. The choice depends on needs for accuracy vs simplicity.

Real-World System Design: Twitter/X Timeline

As an example, consider designing a **Twitter/X-like timeline** (followers/followees feed):

- **Tweets and Storage:** Tweets (messages) are first stored in a durable database (e.g. Cassandra, HBase). In one design, the Tweet Ingestion Service writes each tweet to Cassandra [58]. This ensures all tweets are persisted.

- **Event Pipeline:** After storing, the service emits an **event** (e.g. to Kafka) with the tweet ID and author ID [58] [59]. Kafka (or similar queue) decouples ingestion from delivery. A separate **Tweet Processor** consumes these events and decides how to distribute the tweet.

- **Fan-out on Write:** For each new tweet, the processor looks up all *followers* of the author (via the social graph service/Flock). It then **fans out** the tweet by writing the tweet ID into each follower's home timeline. In practice, Twitter pipelines these writes (e.g. 4K at a time) into a huge in-memory store [60]. Each follower's timeline (a list of tweet IDs) is kept in a Redis cluster. Twitter's blog notes: "Your home timeline sits in a Redis cluster and has a maximum of 800 entries" [61]. So active timelines live in Redis RAM for fast reads.

- **Celebrities and Fan-out on Read:** Users with millions of followers pose a challenge ("giants" like celebrities). Pushing a tweet to millions of timelines on every write is slow and resource-intensive. Twitter found that for these outliers, they sometimes *delay* or even handle fans-out on *read* instead. In other words, they pre-calc most timelines, but for very high-fanout users they may generate parts of the timeline on demand. One summary: "Large fanouts (celebrities) slow write path; Twitter is shifting to do more work on reads for high-value users" [62]. In another guide: "only cache tweets from non-famous users… tweets by famous/celebrity users will be immediately notified" (implying they push only to live sessions) [63].

- **Caching & Storage:** The timeline Redis cluster is sharded by user ID and has replicas (each tweet is replicated 3× across machines) [64] for fault-tolerance. When a user requests their timeline (Pull API), it's typically an O(1) lookup in Redis for the precomputed list [65]. If a user has fallen out of the Redis cache (e.g. long offline), the system "reconstructs" their timeline from followers on the fly [66].

- **Use of Kafka, Redis, Sharding:** This design uses Kafka for reliable event delivery (the firehose) [59], Redis as an in-memory timeline store [61] [63], and a sharded social-graph (FlockDB) for follower lists [64]. Sharding is implicit: Cassandra (tweet store) is distributed; Redis is a distributed cluster; social graph is partitioned (e.g. by user ID).

- **Scaling Patterns:** The read/write ratio is high (read >> write). By doing fan-out on write, read operations become very fast (just read from Redis). This matches Twitter's observation: ~300K QPS on reads vs ~4K QPS on writes [67].

This example shows typical trade-offs: huge fan-out on write speeds up reads, but can overload writes for super-users [62] [67]. Using Redis caches and streaming systems (Kafka) is key to achieving scale.

Architecture Thinking & Interview Preparation

Trade-offs

System design is full of trade-offs. Key examples:

- **Consistency vs Availability (CAP):** During partitions, you can choose to stay available with possibly stale data, or stay consistent but reject some requests [24]. Real systems pick based on needs.

- **Latency vs Throughput:** Tuning for lowest latency (e.g. synchronous replication) can reduce throughput. Async replication boosts throughput but adds lag.
- **Cost vs Performance:** More performance (throughput, availability) usually costs more. For example, to achieve 5-nines uptime you might double or triple infrastructure cost. As one note says, " Availability vs Cost: more resources and replication increase availability but also cost" [3] . Good architecture finds balance (e.g., reserve expensive redundancy for critical parts).
- **Simplicity vs Features:** A simple monolith is cheap to build, but a complex microservices setup offers more scaling and flexibility. Choose simplicity early unless clear reasons demand complexity.

Being aware of these helps in interviews: when proposing a solution, always mention what you gain and what you might sacrifice.

Failure Scenarios

Think through **what can go wrong** and how the system responds:

- **Server crash:** Ensure multiple instances (load-balanced) so traffic shifts if one dies. Use health checks and auto-restart. E.g. stateless web servers mean you can kill one without data loss.
- **Database failure:** Use replication. If a master DB fails, promote a replica.
- **Network partition:** Design for partition tolerance (retry logic, circuit breakers). For example, a timeout on a service call and fallback strategy (maybe cached data) can keep the user happy.
- **Partial outage:** Maybe a cache cluster or region goes down. Use multiple availability zones or a multi-region setup. Also, degrade gracefully (e.g. read-only mode if writes fail).
- In general, assume components fail. Build redundancy and design idempotent, retryable workflows.

Idempotency

An idempotent operation can be applied multiple times without changing the result beyond the initial application [68] . In distributed systems, this is crucial: messages or requests may be retried or duplicated due to timeouts or errors. Ensuring operations are idempotent means duplicates don't cause harm. For example, a `GET` or `PUT` HTTP request should be idempotent by spec [69] (repeating a `PUT` with the same data yields the same state). Payment charges are a classic example: if the user clicks "pay" twice accidentally, the system should recognize the duplicate and only charge once [70] . Often we implement this by checking if a transaction ID was seen before before performing the action [71] . In summary, design APIs and services so that retries (due to timeouts or errors) don't result in double side-effects.

Bottleneck Analysis

A **bottleneck** is a point in the system that limits overall performance [72] . For example, if your app can handle 10,000 requests/sec in total, but the database can only handle 1,000 writes/sec, the DB is the bottleneck. Identifying bottlenecks involves profiling and monitoring (CPU, memory, disk, network usage) [73] . Common bottlenecks include CPU saturation (slow code), memory limits (garbage collection pauses), network congestion, or storage I/O limits [74] [75] . Once identified, you mitigate by scaling that component (e.g. database replication/sharding, adding cache, optimizing code) or offloading work (e.g. moving heavy processing to background jobs). In interviews, walking through a bottleneck often shows deep understanding: find the "slowest link" and propose targeted fixes.

Scaling from 1 → 1M → 100M Users

Systems evolve in stages as usage grows:

1. **Prototype / Few Users:** Start with a simple monolithic setup (single server + database). This is easy to build and deploy quickly.
2. **Tens of Thousands:** When traffic increases, introduce a load balancer with multiple application servers behind it. Split the database from the app so each can scale (App and DB tiers) [76] . Use read-replicas on the DB to offload read load [77] .
3. **Hundreds of Thousands:** Add caching (in-memory caches, Redis) for hot data and a CDN for static assets. Implement asynchronous processing (message queues) for non-critical tasks. Introduce microservices to separate major features (so teams can work independently and scale differently).
4. **Millions:** Migrate to containerized deployments (Kubernetes) and geographically distributed data centers. Use distributed databases (sharded, multi-region) and global CDNs. Emphasize automation, monitoring, and fault tolerance. One roadmap suggests: *Monolith → Load Balancer + multiple servers → Database replication → Caching & CDN → Microservices & distributed DBs* [78] .

Throughout, security, logging, and cost-awareness must be built in. In a design interview, articulating this step-by-step scaling plan (with concrete architectural changes at each phase) shows solid big-picture thinking.

Diagram Guidance: C4 Model

The **C4 model** is a modern approach to drawing architecture diagrams [9] . It uses four "zoom levels":

- **Context diagram:** Shows the system's scope (the system under discussion) in its environment. Include major external users and systems and how they interact with your system [79] . This is high-level and uses just a few boxes (user, system, external services).
- **Container diagram:** Zoom in to show the major "containers" inside your system. A container can be a web app, database, mobile app, or microservice [80] . Draw how these boxes connect (e.g. web app → database, API → microservice). Typical containers: load balancer, API gateway, app servers, database instances, cache cluster, message queue, etc.
- **Component diagram:** Pick one container (say, the web application) and break it into components or modules (libraries, controllers, business modules) [81] . Show interactions between these components.
- **Code (Class) diagram:** (Optional) For a complex component, show classes and relationships.

When drawing system diagrams (for interviews or docs), use clear boxes for each element. Label connections with protocols (HTTP, SQL, etc.). Always include at least a load balancer, API gateway (if used), web servers or microservices, databases, caches, and queues as needed. For example:

```
[Client] --> (Load Balancer) --> [Web Service Servers] --> [Database]
                                |--> [Cache (Redis)]
                                |--> [Message Queue]
```

Here “[]” are components, “()” is an infrastructure element. Use arrows to show data flow. The C4 approach ensures your diagram is organized by level of detail, which makes it easier to discuss different views of the system.

Examples of components to include: Load balancer, API gateway, web/app servers, service nodes, database(s), cache(s), message broker(s). Also note external dependencies (3rd-party services) and clients (web, mobile). A good diagram has about 5–10 boxes at each level – enough detail to be informative but not overwhelming. Finally, always be prepared to walk through the diagram step-by-step in an interview, explaining each component’s role and interactions.

Sources: Authoritative references on architecture and distributed systems [1] [2] [5] [14] [19] [24] [82] [31] [38] [40] [42] [43] [45] [47] [49] [51] [53] [61] [58] [68] [72] [78].

---

[1] Systems architecture - Wikipedia
https://en.wikipedia.org/wiki/Systems_architecture

[2] System Architecture: Why It Matters, Why There’s No One-Size-Fits-All, and Cross-Architecture Compilations | by Synipers | One Shot | Medium
https://medium.com/@synipers/system-architecture-why-it-matters-why-theres-no-one-size-fits-all-and-cross-architecture-1476081ea7b4

[3] [4] [29] System Design Quality Attributes Clearly Explained
https://blog.levelupcoding.com/p/system-design-quality-attributes

[5] [6] [7] 4+1 architectural view model - Wikipedia
https://en.wikipedia.org/wiki/4%2B1_architectural_view_model

[8] What Is a Data Architecture? | IBM
https://www.ibm.com/think/topics/data-architecture

[9] [79] [80] [81] From Context to Code: Mastering Software Architecture with the C4 Model. | by Roland N. Mweze | Medium
https://medium.com/@manfulmwez/from-context-to-code-mastering-software-architecture-with-the-c4-model-87eaef834423

[10] Client-Server Model - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/client-server-model/

[11] What is an API? Application programming interface explained
https://www.wrike.com/blog/what-is-an-api/

[12] [13] Containers vs VM - Difference Between Deployment Technologies - AWS
https://aws.amazon.com/compare/the-difference-between-containers-and-virtual-machines/

[14] Domain Name System - Wikipedia
https://en.wikipedia.org/wiki/Domain_Name_System

[15] What is a content delivery network (CDN)? | How do CDNs work? | Cloudflare
https://www.cloudflare.com/learning/cdn/what-is-a-cdn/

[16] [17] [18] SQL vs. NoSQL Databases: What's the Difference? | IBM
https://www.ibm.com/think/topics/sql-vs-nosql

[19] [20] ACID vs BASE - by Nikki Siapno
https://blog.levelupcoding.com/p/acid-vs-base

[21] [22] Database indexing basics: how indexes make queries faster
https://upsun.com/blog/basics-of-database-indexing/

[23] Database normalization - Wikipedia
https://en.wikipedia.org/wiki/Database_normalization

[24] CAP theorem - Wikipedia
https://en.wikipedia.org/wiki/CAP_theorem

[25] [26] [27] System Design Principle: CAP Theorem | by Wahidyan Kresna Fridayoka (Yoka) | AyoKoding | Medium
https://medium.com/ayokoding/system-design-principle-cap-theorem-f832915a1ee7

[28] [82] Eventual vs Strong Consistency in Distributed Databases - GeeksforGeeks
https://www.geeksforgeeks.org/dbms/eventual-vs-strong-consistency-in-distributed-databases/

[30] Microservices vs. monolithic architecture | Atlassian
https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith

[31] Layer 4 vs Layer 7 Load Balancing | A10 Networks
https://www.a10networks.com/glossary/how-do-layer-4-and-layer-7-load-balancing-differ/

[32] [33] [34] [35] [36] Caching Essentials: Types, Strategies, and Best Practices | by Dr. Yaroslav Zhbankov | Medium
https://medium.com/@yaroslavzhbankov/caching-essentials-types-strategies-and-best-practices-459493cc47d9

[37] Cache Invalidation and the Methods to Invalidate Cache - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/cache-invalidation-and-the-methods-to-invalidate-cache/

[38] [39] Kafka vs RabbitMQ: Key Differences & When to Use Each | DataCamp
https://www.datacamp.com/blog/kafka-vs-rabbitmq

[40] [41] Database Sharding vs Replication - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/difference-between-database-sharding-and-replication/

[42] Pattern: Saga
https://microservices.io/patterns/data/saga.html

[43] Raft (algorithm) - Wikipedia
https://en.wikipedia.org/wiki/Raft_(algorithm)

[44] [45] [46] What Is Event-Driven Architecture? | IBM
https://www.ibm.com/think/topics/event-driven-architecture

[47] [48] Difference Between CQRS and Event Sourcing - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/difference-between-cqrs-and-event-sourcing/

[49] [50] Beginner's Guide: What is Istio Service Mesh? | Kong Inc.
https://konghq.com/blog/learning-center/what-is-istio-service-mesh

[51] [52] Batch vs Stream Processing: When to Use Each and Why It Matters | DataCamp
https://www.datacamp.com/blog/batch-vs-stream-processing

53 54 55 56 57 Lambda Architecture vs. Kappa Architecture in System Design - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/lambda-architecture-vs-kappa-architecture-in-system-design/

58 59 63 System Design of X/Twitter. X/Twitter is a short messaging... | by Santosh P. | Medium
https://medium.com/@lazygeek78/design-of-x-twitter-7233483b5d31

60 61 62 64 65 66 67 The Architecture Twitter Uses to Deal with 150M Active Users, 300K QPS, a 22 MB/S Firehose, and Send Tweets in Under 5 Seconds - High Scalability -
https://highscalability.com/the-architecture-twitter-uses-to-deal-with-150m-active-users/

68 69 70 71 Idempotence & Idempotent Design in IT/Tech Systems | Splunk
https://www.splunk.com/en_us/blog/learn/idempotent-design.html

72 73 74 75 Bottleneck Conditions Identification in System Design - GeeksforGeeks
https://www.geeksforgeeks.org/system-design/bottleneck-conditions-identification-in-system-design/

76 77 78 System design Scale from ZERO to MILLION Users | by Sumit sharma | Medium
https://medium.com/@sumit.dev2148/system-design-scale-from-zero-to-million-users-e5af1bcc4dcb